GUIDE 3 OF 8 · AI LEARNING SERIES

Generative AI — Foundations

LLMs, Transformers, Prompting, Fine-tuning, Multimodal — from first principles

01 What is Generative AI?

Generative AI refers to AI systems that produce new content — text, images, code, audio, video — rather than just classifying or predicting from existing data. The content is generated by learning patterns from vast training datasets.

Type	What It Generates	Key Models
Large Language Models (LLMs)	Text, code, structured data	GPT-4o, Claude 3.5, Llama 3, Mistral, Gemini
Image Generation	Images from text descriptions	DALL-E 3, Stable Diffusion, Midjourney, Flux
Audio Generation	Speech, music, sound effects	ElevenLabs, Whisper, MusicGen, Bark
Video Generation	Video clips from text/image	Sora, Runway Gen-3, Pika, Kling
Code Generation	Code in any language	GitHub Copilot, Claude, CodeLlama, Qwen Coder
Multi-modal	Text + Image + Audio together	GPT-4o, Claude 3.5 Sonnet, Gemini 1.5 Pro

02 How LLMs Work — The Transformer

All modern LLMs are built on the **Transformer** architecture (Attention Is All You Need, Vaswani et al. 2017). Understanding it at a high level makes everything else click.

Step 1: Tokenization

Text is split into **tokens** — roughly word-pieces. "unhappiness" → ["un", "happ", "iness"]. Every unique token has a numerical ID. The model never sees raw text — only token IDs.

> **EXAMPLE**
>
> "Hello world" → [15496, 995]. GPT-4 has ~100K tokens. Claude uses ~100K tokens. 1 token ≈ 0.75 words on average.

Step 2: Embeddings

Each token ID is converted to a high-dimensional vector (e.g., 4096 dimensions for large models). This is different from RAG embeddings — these are internal representations learned during training.

Step 3: Self-Attention — The Core Innovation

For each token, the model computes how much it should "attend to" every other token in the sequence. This captures long-range dependencies: "The bank by the river was flooded" — "bank" attends strongly to "river" and "flooded" to understand it means a riverbank, not a financial institution.

```
Attention(Q, K, V) = softmax(QK^T / sqrt(d_k)) × V
```

Q=Queries, K=Keys, V=Values — all linear projections of the input. d_k = key dimension.

INTUITION

Think of attention as: for each word, look at every other word and decide "how relevant is that word to understanding THIS word?" Weights sum to 1. High weight = strong relevance.

Step 4: Next-Token Prediction

LLMs are trained to predict the next token given all previous tokens. After billions of examples, the model learns grammar, facts, reasoning, and style — all as a side effect of this simple objective.

At inference time: You provide a prompt (token sequence). The model outputs a probability distribution over all ~100K tokens. Sample from this distribution to pick the next token. Repeat until done.

Key LLM Concepts

Concept	What It Means	Why It Matters
Context Window	Max tokens model can process at once	Longer context = more info, but costs more. GPT-4o: 128K. Claude: 200K.
Temperature	Randomness in token sampling (0=deterministic, 2=creative)	Low temp for factual tasks, higher for creative writing.
Top-P (nucleus)	Sample from top tokens whose cumulative prob >= P	Controls diversity. Usually 0.9-0.95.
Top-K	Only consider top K tokens at each step	Hard cap on diversity. Often K=40-50.
System Prompt	Instructions that persist across the conversation	Define model role, constraints, format, persona.
Few-Shot Prompting	Provide examples in the prompt	Show don't tell. 3-5 examples often doubles quality.
Chain-of-Thought	Ask model to reason step-by-step before answering	"Let's think step by step" improves reasoning significantly.

03 Prompt Engineering

Prompt engineering is the art of communicating clearly with an LLM to get the output you want. It's the highest-ROI skill in applied GenAI.

Core Techniques

Technique	How To Use It	Example

Zero-Shot	Just ask. No examples.	"Classify this review as positive or negative: ..."
Few-Shot	Provide 2-5 examples of input→output	"Review: Great! → Positive Review: Terrible → Negative Review: Okay → ..."
Chain-of-Thought (CoT)	Ask model to show its reasoning	"Think step by step. What is 17% of 340?"
Role Prompting	Assign an expert persona	"You are a senior Python engineer. Review this code for security issues."
Output Format Control	Specify exact output structure	"Respond ONLY as JSON: {'sentiment': ..., 'confidence': ...}"
Negative Examples	Tell model what NOT to do	"Do not use bullet points. Do not exceed 3 sentences."
Self-Consistency	Generate multiple answers, take majority	Run same prompt 5x, aggregate answers.
ReAct	Reason then Act in a loop	Thought → Action → Observation → Thought → ...

04 Fine-Tuning vs RAG vs Prompting

Three ways to adapt an LLM to your domain. Choosing right saves enormous cost and effort.

Approach	What It Does	Cost	When To Use
Prompt Engineering	Guide behavior via instructions	Zero	Always try first. Solves 80% of cases.
RAG	Inject relevant docs at query time	Low (infra)	Domain knowledge, up-to-date info, private data.
Fine-Tuning	Update model weights on your data	High (GPU)	Specific style/format, task specialization, speed optimization.
Full Pre-training	Train from scratch on domain corpus	Very High	Highly specialized domain (medical, legal, code).

RULE OF THUMB

Try prompting first. If results are good but lack domain knowledge → add RAG. If you need consistent style/behavior regardless of prompt → fine-tune. Never jump to fine-tuning without trying RAG first.

05 Key LLMs — Comparison

Model	Provider	Context	Best For
GPT-4o	OpenAI	128K	General purpose, vision, function calling. Best ecosystem.
GPT-4o-mini	OpenAI	128K	Cost-effective. 90% quality at 10% price. RAG generation.
Claude 3.5 Sonnet	Anthropic	200K	Coding, analysis, long-form. Best reasoning. Large context.
Claude 3 Haiku	Anthropic	200K	Fast and cheap. Good for high-volume RAG.

Llama 3.1 (70B)	Meta (open)	128K	Best open-weight model. Run locally or on cloud.
Mistral (7B/8x7B)	Mistral (open)	32K	Efficient, fast, strong coding. Great for local RAG.
Qwen 2.5 Coder	Alibaba (open)	128K	Best open-source code model. Your internal LLM.
Gemini 1.5 Pro	Google	1M	Enormous context window. Multi-modal. Best for long docs.
Gemma 2 (9B/27B)	Google (open)	8K	Compact, deployable, good for edge/local inference.

06 GenAI Tools Ecosystem

Category	Tool	Use It For
LLM APIs	OpenAI Python SDK	pip install openai — call GPT-4o, embeddings, DALL-E
LLM APIs	Anthropic SDK	pip install anthropic — call Claude models
LLM APIs	LiteLLM	One interface for 100+ LLM providers. Drop-in OpenAI replacement.
Local LLMs	Ollama	Run Llama, Mistral, Qwen locally. ollama run llama3
Local LLMs	LM Studio	GUI for local LLMs. Download and run without coding.
Frameworks	LangChain	Chains, agents, RAG, tools. Most ecosystem.
Frameworks	LlamaIndex	Document indexing, RAG, data connectors.
Frameworks	DSPy	Programmatic prompting — optimize prompts via gradient descent.
Evaluation	PromptFoo	Test prompts against test cases. CI/CD for prompts.
Image Gen	diffusers (HuggingFace)	Run Stable Diffusion locally. pip install diffusers.
Speech	Whisper (OpenAI)	State-of-the-art speech-to-text. Local or API.
Observability	LangSmith	Trace, debug, and evaluate LLM calls.